**Intelligent Document Reference: Local-First Retrieval Augmented Generation System For Citation-Grounded Question Answering**

CSC398: Building AI/ML Systems

Dev Patel

Ishan Singh

Kimberly Michela

Sharat Krishnan

Zoya Fatima

University of Toronto

Repository: *Intelligent-Document-Reference-Winter2026*

April 6th, 2026

# Table of Contents

**Executive Summary**.....3
**1. Project Background & Context**.....3
1.1 Project Overview.....3
1.2 Technical Context.....4
1.3 Open-Source and Community Context.....4
**2. Problem Definition**.....5
2.1 Core Problem.....5
2.2 System Challenges.....5
2.3 Stakeholder Impact.....6
**3. Proposed Solution**.....6
3.1 Solution Overview.....6
3.2 Design Rationale.....6
3.3 Expected Benefits.....7
**4. Technical Implementation Summary**.....8
4.1 High-Level System Architecture & Technology Stack.....8
4.2 Indexing & Ingestions Pipeline.....9
4.3 Retrieval Pipeline.....10
4.4 Tradeoffs.....11
4.5 Validation & Evaluation.....12
**5. Ethical, Social, and Community Implications**.....13
5.1 Data Governance and Licensing.....13
5.2 Model Transparency and Explainability.....14
5.3 Bias and Representation.....14
5.4 Over-Reliance and Social Responsibility.....14
5.5 Accessibility and the Equity Gap.....15
5.6 Open-Source Ecosystem Impact.....15
**6. Recommendations & Future Work**.....15
6.1 Technical Recommendations.....15
6.2 Known Limitations.....16
6.3 Future Work.....16
**7. Conclusion**.....17
**References**.....18

# Executive Summary

Intelligent Document Reference is a local-first Retrieval-Augmented Generation (RAG) system designed for citation-grounded question answering over a user-owned document collection. The system addresses the critical gap in the existing tools where standard LLMs hallucinate when queried without grounded context and any native filesystem tools like macOS spotlight are limited to only keyword matching and cannot query across documents. To solve this, the project implements a hybrid semantic and lexical retrieval pipeline backed by SQLite extended with sqlite-vec, providing efficient vector search without cloud dependency. Documents are continuously ingested and indexed through an asynchronous file-watcher pipeline that supports structured, unstructured, and multimodal formats. A FastAPI backend and React/Vite frontend give a familiar interface where users can query their files and receive answers that are grounded in source citations.

Benchmark evaluation across 16 dataset suites demonstrated strong retrieval performance and high LLM-judge response quality, confirming that the system produces semantically correct and verifiable answers. Key recommendations for future work include adopting approximate nearest-neighbour indexing for larger collections, cross-encoder reranking and multi-user collaboration support.

*AI citation: https://chatgpt.com/share/69d47f51-b694-8333-a4f8-e360dcaabe03*

# 1. Project Background & Context

## 1.1 Project Overview

Intelligent Document Reference bridges the gap between static file storage and intelligent information synthesis. Its primary goal is a seamless pipeline for document ingestion and citation-grounded question answering. Moving beyond the severe limitations of traditional keyword search, this system understands meaning across file contents, filenames, and metadata, connecting related information scattered across multiple documents.

Users require absolute confidence that the LLM generated information is both accurate and directly traceable to their original documents. By strictly grounding the generative process in user-provided local data, this system mitigates the hallucination risks inherent in generalized language models, ensuring that all generated responses are verifiable and backed by explicit source citations.

Ultimately, given a specific user query, the system is required to retrieve the most relevant files and generate clear answers that are strictly supported by explicit citations to the source documents.

## 1.2 Technical Context

The underlying architecture is built on an optimized Retrieval-Augmented Generation (RAG) framework, driven by vector search and semantic retrieval. At the persistence layer, the system usesan embedded SQLite database augmented with the sqlite-vec extension. This provides a lightweight, highly efficient local vector database capable of rapid k-nearest neighbour (kNN) similarity searches on-device, entirely bypassing the operational overhead and latency of cloud-based vector stores.

The application infrastructure leverages a FastAPI backend paired with a React/Vite frontend. To ensure data remains current, the file ingestion workflow continuously monitors user directories using a Python-based file watcher that detects creation, modification, deletion, and renaming events while debouncing rapid updates. Asynchronous workers then execute change-detection logic by comparing current file metadata against the last-indexed state. This allows the system to perform fast, incremental reindexing, skipping unmodified files entirely to avoid redundant processing. The extraction pipeline is designed to parse heterogeneous and multimodal file formats, including structured data (CSV, JSON, Excel), unstructured documents (DOCX, PDF), and images (PNG, JPG).

Users can route requests between higher-capacity cloud APIs and local implementations, such as utilizing Ollama as an on-device embedding backend. This design gives users complete control over the trade-offs between processing speed, model capabilities, and absolute data privacy.

## 1.3 Open-Source and Community Context

This project aligns with the principles of the Linux Foundation by focusing on the benefits of local-first AI systems, prioritizing data sovereignty, security, and user autonomy. As an open-source tool, it guarantees reproducibility and developer accessibility, allowing individuals and organizations to audit the code or deploy the system entirely offline. This offline capability ensures users can completely avoid network API calls, keeping all sensitive data strictly on-device for maximum privacy and control. However, recognizing that different users have varying computational resources, the system is designed to support a broad range of preferences. It provides optional cloud-assisted configurations that enable higher-capacity models, allowing users to actively choose the trade-off between privacy, processing speed, and retrieval accuracy that best fits their specific needs.

The project's modular architecture is explicitly designed for long-term community maintainability and developer contribution. The pipeline's abstraction layers allow external open-source contributors to seamlessly integrate new multi-modal input parsers, test custom embedding models, or swap out Vision Language Models (VLMs) overhauling the core database schema or file-watching logic.

By providing clear, documented endpoints for these tools, the project acts as a foundational, interoperable service. Ultimately, this approach democratizes access to intelligent, verifiable document analysis while fostering a collaborative open-source ecosystem focused on transparent, privacy-preserving generative AI.

*AI citation: https://chatgpt.com/share/69d2d7fc-9950-8329-b4bb-46953f4a32f3*

# 2. Problem Definition

## 2.1 Core Problem

While Large Language Models (LLMs) have revolutionized natural language processing, they frequently hallucinate when generating information without grounded context. For individuals and professionals, a massive gap exists between the capabilities of standard LLMs and their ability to accurately reference large, unorganized sets of local documents. Existing native filesystem solutions, such as macOS Spotlight or Windows Search, are restricted to metadata filtering and exact keyword matching. They cannot process semantic intent or intuitively connect related concepts across disparate files. Conversely, attempting to solve this by directly feeding local data into an LLM is technically unfeasible. Standard models have strict context window limits, making it impossible to include an entire filesystem's contents in a single prompt.

Furthermore, users are often forced to choose between highly capable cloud-based models that compromise data privacy and less capable local systems. The core problem is the lack of a trustworthy, privacy-respecting application that leverages RAG to overcome these dual bottlenecks. By maintaining all local data in a separate, optimized vector store and dynamically fetching the precise semantic chunks needed for inference, such a system would provide a deep understanding of local file systems while producing highly auditable AI answers backed by explicit source citations.

## 2.2 System Challenges

Building an intelligent, local-first document reference architecture introduces several critical system challenges:

- **Efficient local document ingestion**: The system must continuously monitor the file system for modifications and perform fast, incremental indexing. Reprocessing an entire local directory for every minor change is computationally unfeasible, requiring sophisticated change-detection logic.
- **Chunking and semantic indexing**: To prepare data for RAG, documents must be dynamically segmented using optimized chunking algorithms before being embedded into a vector database.
- **Citation alignment between retrieved chunks and generated answers**: To establish trust, the system must tightly align the retrieved vector chunks with the generated response, ensuring that the model explicitly and correctly cites the source files and chunk identifiers rather than producing generic answers.
- **Latency vs retrieval quality**: Maintaining a fast query response pipeline is challenging when routing semantic searches through local embedding and inference models. The system must balance the computational cost of deep retrieval against end-to-end user latency.
- **Multi-format file support**: Modern file systems are highly heterogeneous. Extracting semantic value requires parsing not just plaintext or structured files (like CSVs and JSONs), but also unstructured documents (PDFs, DOCX) and multimodal inputs like images via Vision Language Models (VLMs).

### 2.3 Stakeholder Impact

The inability to semantically navigate local file systems creates significant friction across multiple user domains:

- **Students and researchers**: Academic environments produce chaotic file systems with non-descriptively named files, lecture slides, and scanned handwritten notes. Without semantic search, these users waste critical time manually browsing files to locate specific methodologies or concepts.
- **Software Developers**: Developers frequently context-switch across large legacy repositories, JSON configurations, and scattered markdown files. Standard IDE searches fail when precise variable names are unknown, whereas a semantic tool can isolate logical implementations regardless of obscure naming conventions.
- **Knowledge Workers (Finance, Legal, HR)**: Professionals overseeing extensive collections of structured spreadsheets and unstructured PDFs struggle to manually track changes across versions over time. They require systems that can synthesize meaning and surface financial or contractual changes automatically.
- **Users Requiring Auditable AI**: For individuals prioritizing data ownership or handling sensitive information, the ability to operate entirely on-device without network API calls is a strict requirement. These open-source contributors and privacy-conscious users require explicit citations to verify the AI's logic without relinquishing data to the cloud.

*AI Citation: https://chatgpt.com/share/69d2d8af-a8cc-8332-9728-7eb144fa2771*

## 3. Proposed Solution

### 3.1 Solution Overview

The system implements a local-first RAG pipeline for efficient, citation-grounded document query. Documents are ingested, chunked, and embedded into vector representations to enable fast and accurate retrieval. The web application interface provides seamless interaction with the user's documents and query workflow. Users with limited computing resources can optionally use remote API-based models (e.g., Gemini) to maintain accessibility without sacrificing functionality.

### 3.2 Design Rationale

The design prioritizes:

- **Robust Ingestion:** Documents are parsed into text, cleaned, and segmented into chunks sized to balance semantic coherence and retrieval efficiency. Modality-aware handlers support accurate ingestion and indexing of different file types.

- **Cosine Similarity:** Similarity between query and chunk vectors is computed using metrics such as cosine similarity, a standard similarity measure in semantic search that prioritizes vector orientation over magnitude. (Bianchini, F., 2025)
- **kNN Search over sqlite-vec:** Retrieval uses exact k-nearest neighbour (kNN) search over dense vector embeddings stored in sqlite-vec. Each document chunk and user query is embedded into a high-dimensional space, and similarity scores are computed to identify the top-k closest chunks. This exact kNN search retrieves contextually relevant information even when wording differs, forming a core component of dense retrieval in RAG frameworks (Brown, Roman, & Devereux, 2025).
- **Hybrid Retrieval Strategy:** The system combines semantic and lexical retrieval to maximize recall, as hybrid approaches have been shown to outperform purely semantic or lexical methods (Weinberg, A.).
- **Reciprocal Rank Fusion (RRF):** Retrieved results from both semantic and lexical search are aggregated using RRF, improving ranking robustness and ensuring more consistent output quality (Bruch, Gai, & Ingber).
- **Privacy**: Local processing keeps sensitive documents on the user's device.
- **Efficiency**: Local models reduce cost and accelerate testing.
- **Immediate Updates**: A file watcher detects changes and re-indexes documents automatically.
- **Transparency**: Citation-grounded answers enhance trust.
- **Flexibility**: Users can switch between local and cloud models depending on performance or budget.

### 3.3 Expected Benefits

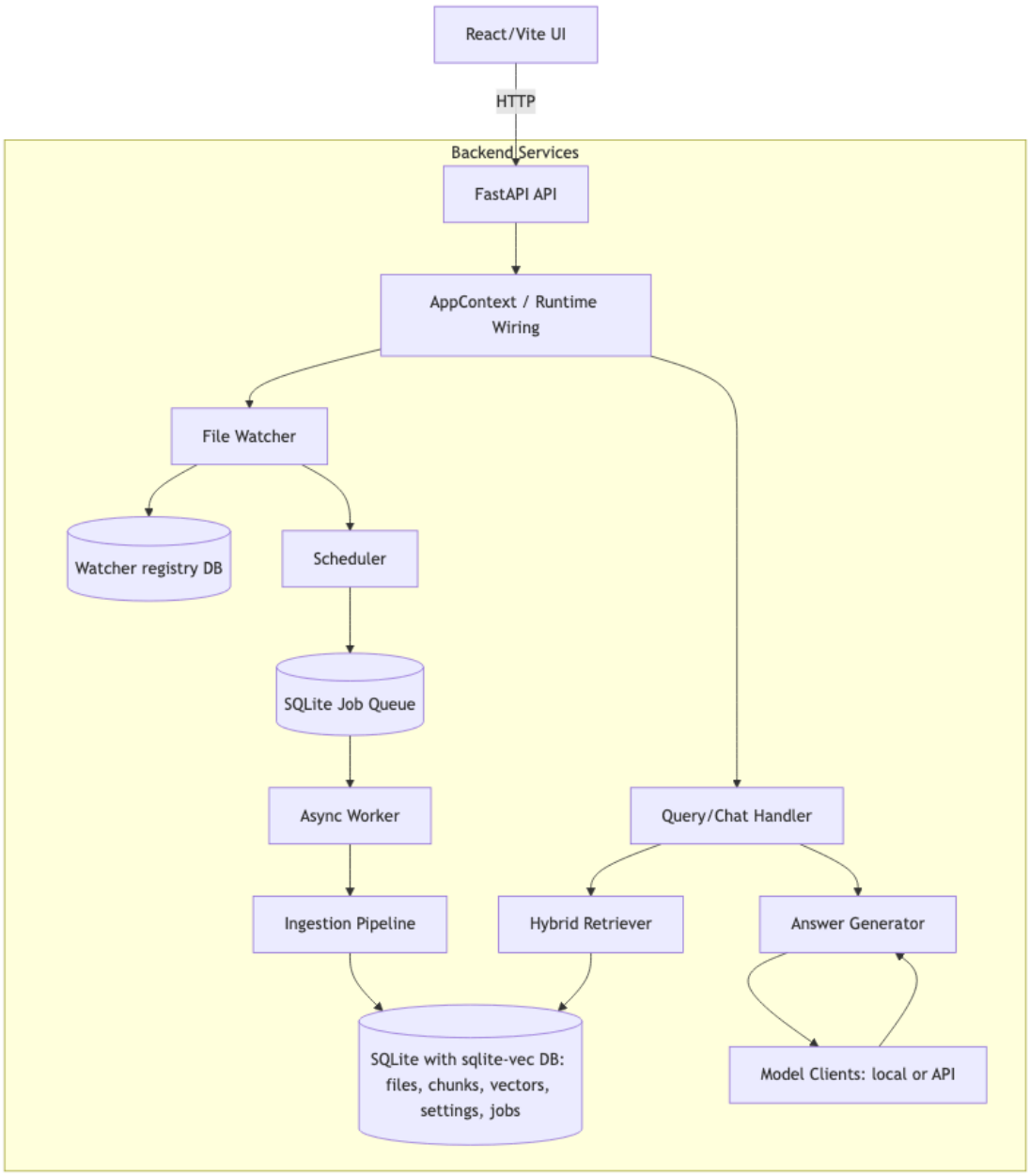
- **Verifiable Outputs**: Citation-linked responses enable users to confirm the accuracy of retrieved information.
- **User Trust and Privacy**: Local processing ensures sensitive documents remain on the user's device, reducing exposure to external servers.
- **Immediate Feedback Loop**: The file watcher and automatic re-indexing allow document updates to be reflected instantly, eliminating the need for manual refresh or re-upload.
- **Offline Functionality**: Users can perform queries without an internet connection in local-first mode, ensuring uninterrupted access.
- **Modularity for Experimentation**: Developers or researchers can swap embedding models, retrievers, or LLMs to evaluate performance trade-offs.
- **Cost Optimization**: Users can leverage lower-cost local models for routine queries and switch to premium API-based models for more complex tasks.

This design delivers a flexible, efficient, and trustworthy RAG system, balancing performance, accessibility, and user control while directly addressing the project objectives.

# 4. Technical Implementation Summary

## 4.1 High-Level System Architecture & Technology Stack

The architecture is intentionally split into two pipeline verticals: (1) an asynchronous indexing and ingestion pipeline that facilitates the file indexing process and watches for changes (left-hand side of diagram below), and (2) a retrieval pipeline that handles the query, retrieves matching chunks, and generates the final answer/response (right-hand side of diagram below).

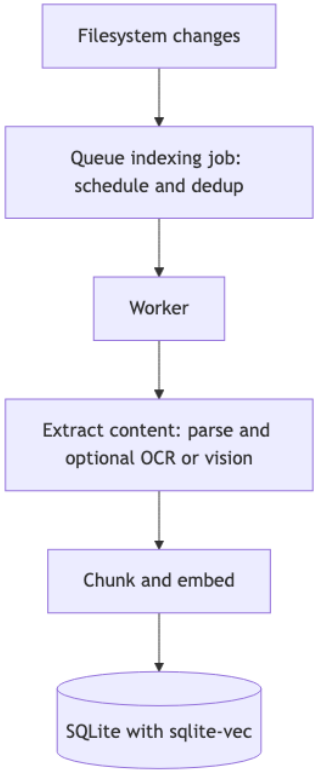



The application is structured as a **FastAPI** backend coupled to a **React/Vite** frontend. FastAPI provides clear request/response contracts (which can be seamlessly validated with Pydantic classes), response streaming (typical for LLM responses), lightweight asynchronous processing (critical for the Python backend with many I/O operations), lifecycle hooks that can initialize and manage long-lived

background services, and is currently the industry standard for many LLM API providers. React/Vite enables an interactive UI with a modern build pipeline, where production assets can be served statically by the backend (removing dependency prerequisites for end-users), while development can rely on a separate development server for rapid iteration.

At the database layer, the system uses **SQLite** extended with **sqlite-vec** as the embedding vector database. This design removes the operational complexity of external vector databases, keeps the system deployable as a single local application, and enables in-process kNN retrieval. The relational schema stores source file metadata (paths, versions, and indexing status) and associates each chunk of extracted content with its semantic embedding. Vector rows are coupled to chunk identifiers to support efficient joins between dense retrieval results and human-readable metadata required for citations.

### 4.2 Indexing & Ingestions Pipeline



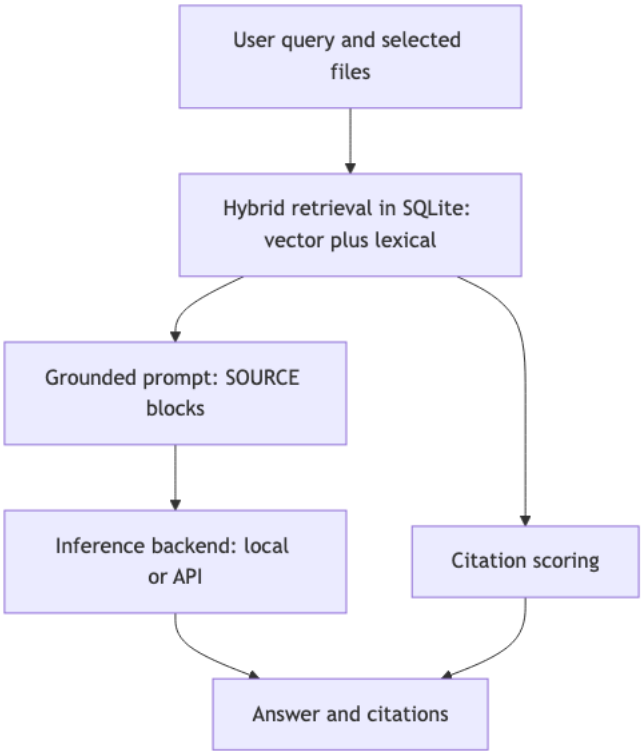

To keep the index synchronized with user files, the system continuously monitors configured directories and files using a Python-based watcher called **watchdog**. The watcher handles file creation, modification, deletion, and move/rename events. Rather than triggering synchronous reindexing on every event, events are normalized into discrete indexing tasks and submitted to a persistent queue that deduplicates work per file path and can bump priority on repeat events. The queue also supports explicit prioritization, allowing user-triggered indexing actions from the web UI to preempt background watcher-driven updates.

Index execution is performed by asynchronous workers that poll the queue and apply a change-detection strategy before performing expensive operations. This change detection compares the

current file state against the last indexed state (including metadata and, when needed, content hashing), and the goal here is to index incrementally and skip any unchanged files to reduce redundant parsing and embedding work.

Finally, during ingestion, files are parsed through modality-aware handlers that support multiple file types. The implementation includes robust handling for structured formats (CSV, JSON, spreadsheets), unstructured documents (DOCX, PDF), and images, along with support for audio files. For visually encoded content (e.g., scanned PDF pages or standalone images), the pipeline uses Optical Character Recognition (OCR) and/or a vision-capable model to describe images and extract text. Extracted content is then chunked (to balance retrieval precision), embedded using the configured embedding backend, and written to the SQLite database with an explicit per-file status (indexed/failed status to enable potential retries).

### 4.3 Retrieval Pipeline




At query time, the backend accepts a user query alongside an explicit file scope (the set of selected files/folders selected by the UI). The retriever enforces this scope as a hard constraint to make answers more predictable in multi-folder scenarios.

Retrieval is hybrid, where first the query is embedded and used to perform a top-k nearest-neighbour search against sqlite-vec embeddings (dense retrieval), and in parallel, the system computes a lightweight lexical score using SQL matching against chunk text and path metadata (lexical retrieval). These ranked candidate lists are then fused using the reciprocal rank fusion algorithm, which

improves robustness when either vector similarity or lexical matching fails in isolation (e.g., identifier-heavy queries, file-name lookups, or semantically ambiguous prompts).

Finally, once the top chunks are selected, the response generator constructs a grounded prompt that provides explicit SOURCE blocks to emphasize the source-of-truth to the LLM. The system then calls the selected inference backend (which supports local runtimes with **Ollama** and external APIs) to generate the final response, and citations are scored separately from the main answer so that the UI can present traceable evidence alongside the generated response.

### 4.4 Tradeoffs

Before the implementation phase, the very first important tradeoff that needed to be considered was the choice of vector database (as this is where all content embeddings will be stored and queried). Since SQLite is the go-to database for local metadata storage, the natural options were to extend SQLite with a vector extension (sqlite-vec) or introduce a new vector datastore that is popular for local filesystems and provides a simpler interface, such as ChormaDB. Hence, the technology stack was decided with proactive performance benchmarks, and below is the output from a script that returns the results:

```
--- Benchmarking SQLITE ---
Ingestion (5000 items): 0.1989s (25133.6 items/s)
Query (avg of 50): 5.49ms

--- Benchmarking CHROMA ---
Ingestion (5000 items): 1.9418s (2574.9 items/s)
Query (avg of 50): 2.02ms
```

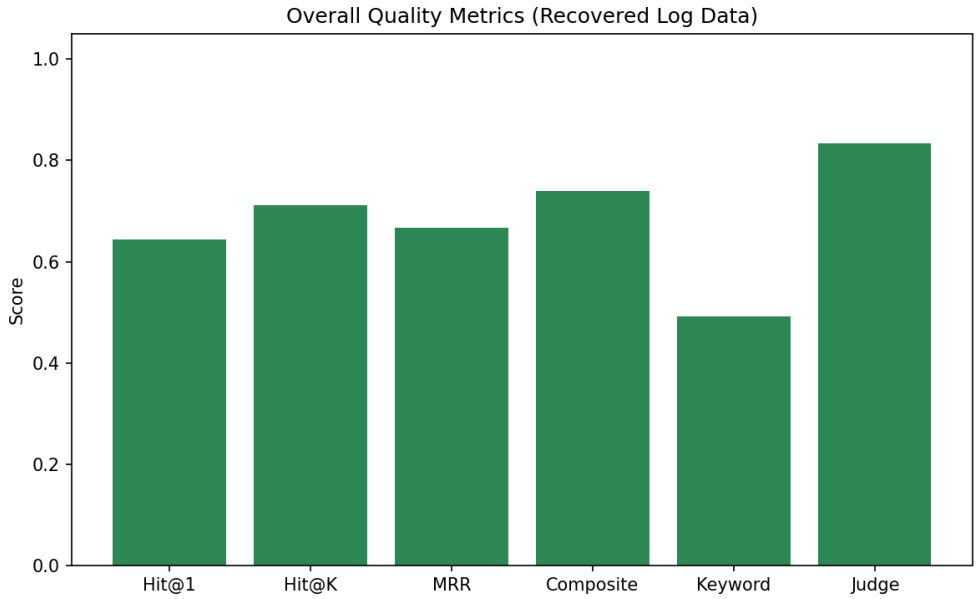
The results suggest that SQLite is almost 10x better for writes, whereas ChromaDB is about 2x better for reads. Proportionally, there will be significantly more indexing jobs as opposed to queries (each file passed for indexing contains multiple chunks, coupled with entire directories almost certainly results in far greater indexing jobs), hence the final tradeoff decision to prioritize ingestion throughput led to the implementation of the sqlite-vec extension as the vector database, which also made integration with metadata database completely seamless (as you can unify the schemas and maintain just one database).

Furthermore, the chosen hybrid retrieval (vector plus lexical) improves robustness on identifier- and filename-heavy queries without introducing external dependencies, but comes at the cost of using heuristics rather than fully learned ranking. And finally, the multimodal extraction (OCR and vision descriptions) broadens the support for multiple file extensions, but introduces an accuracy/cost tradeoff because extraction quality depends on OCR and model behaviour, and also, vision processing can be more expensive than text-only indexing.

### 4.5 Validation & Evaluation

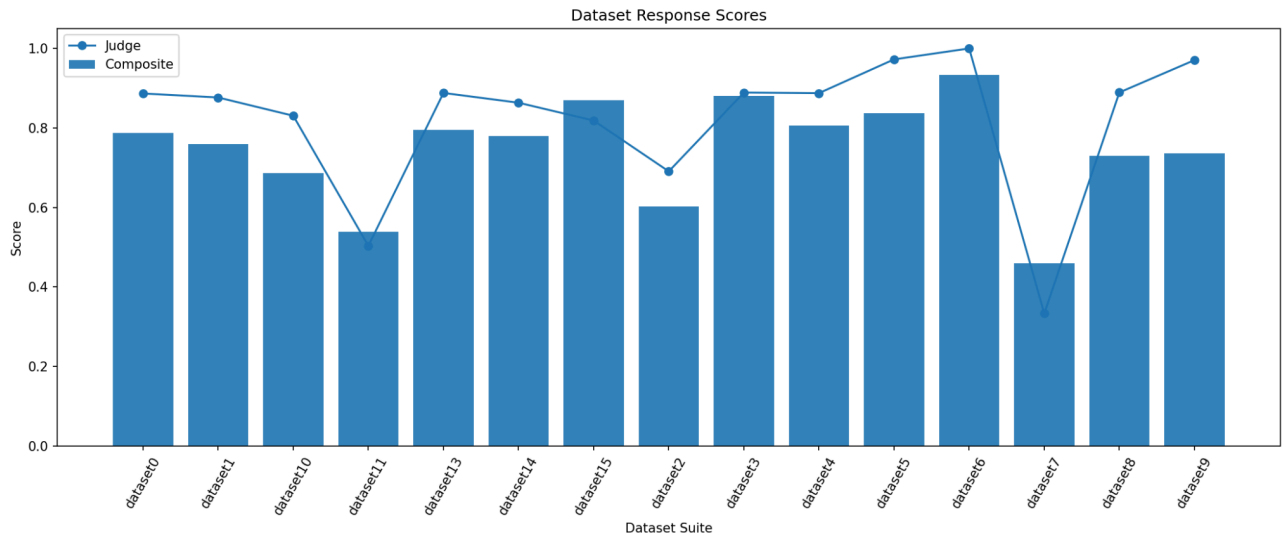
Validation depends on runtime configuration, embedding backends, and the persisted vector schema, so the implementation validates a small set of high-impact rules explicitly. Embedding dimensionality must match the vector table definition (mismatches propagate errors), and provider settings are validated early (e.g., setting invalid API credentials provides an error pop-up). Also, there is validation on changed data, where indexing uses change detection to reindex only modified files, and the retrieval caching is invalidated on stale data to preserve citation correctness.

Evaluation is supported by a benchmark harness that runs prompt suites over curated datasets and computes both retrieval and response-quality metrics on an isolated SQLite database per run. Results are output as structured artifacts (CSVs and JSONs) and visualized with plots:




The plot above summarizes aggregate retrieval and response metrics over the benchmark suite. Hit@1 is a strict top-1 success rate (the expected file is the first result), and Hit@K is a top-K success rate (the expected file appears anywhere in the first K results). MRR (mean reciprocal rank) captures how highly the expected file is ranked by taking 1/rank for each query and averaging across queries. Keyword score is a deterministic coverage metric (fraction of expected keywords found in the generated response), judge score is produced by an LLM-as-a-judge that rates relevance, grounding, and completeness on a 0-1 scale and returns an overall score, and finally the composite score is a weighted combination of the previous metrics.

Analyzing the plot, retrieval metrics (Hit@1/Hit@K/MRR) scored decently and in a similar range, while response scoring shows lower keyword overlap but higher judge and composite scores, and this is consistent with semantically correct answers that are not strictly keyword-aligned (such as paraphrasing).




The plot above summarizes response quality by dataset suite, with the composite score shown as bars and the judge score shown as lines. Because the judge score is intended to reflect semantic correctness (rather than literal keyword), differences between the judge and composite scores can indicate whether failures are primarily semantic (the answer is wrong or ungrounded) or just have to do with wording (the answer is reasonable but uses different phrasing than the expected keywords).

Analyzing the plot, most datasets score highly, with little to no difference in judge and composite score (this is a good sign that suggests no apparent failure in the system). There are some exceptions (such as dataset7) which score relatively lower, however, this is expected given the highly nested structure of the datasets and their intentionally injected discrepancies when curating the datasets.

All in all, these metrics and plots provide a reproducible and multi-angle evaluation that quantifies retrieval correctness and answer quality. The benchmark evaluation across 16 dataset suites demonstrated strong retrieval performance (Hit@K ~0.70, MRR ~0.68) and high LLM-judge response quality (~0.85), and brings confidence that the system behaves correctly and consistently across different datasets and workloads.

# 5. Ethical, Social, and Community Implications

The Intelligent Document Reference System involves more than technical performance; its use of LLMs and document ingestion pipelines creates specific responsibilities regarding data governance, transparency, and equity.

## 5.1 Data Governance and Licensing

While users retain ownership of their documents, the system architecture introduces risks:

- **Third-Party Exposure:** Using cloud-based models requires sending user data to external providers, creating uncertainty about how it is stored, reused, or potentially incorporated into training data.

- **Open-Source Accountability:** Because the system is open source, deployers must understand and communicate trade-offs, such as privacy versus performance, rather than assuming safe defaults. Clear guidance is needed to prevent misuse or accidental data exposure.

### 5.2 Model Transparency and Explainability

The system uses RAG to ground outputs in user-provided text, providing a level of "traceability" through citations.

- **Citation-Based Transparency:** RAG provides citations, allowing users to trace outputs back to sources.

- **Opaque Reasoning:** The internal decision-making of the model, such as how it reconciles conflicting sources or prioritizes information, is not visible to users.

- **Model Variability:** Differences in behaviour between local and cloud models are difficult to explain, which makes evaluation and trust more challenging.

### 5.3 Bias and Representation

Bias enters the system through three primary channels:

- **Document Bias:** Outputs can reflect outdated or skewed information from the source documents.

- **Model Bias:** LLMs carry biases from their training data that can influence results in subtle ways.

- **Pipeline Bias:** Preprocessing steps, such as OCR on complex or low-quality documents, may omit or misinterpret information, compounding biases throughout the system.

### 5.4 Over-Reliance and Social Responsibility

The fluency of LLM responses can lead to "automation bias," where users treat AI summaries as authoritative without verifying the provided citations.

- **Automation Bias:** The confident tone of generated outputs may lead users to accept answers without critical evaluation.

- **High-Stakes Misuse:** Users may rely on the system for domains such as medical or legal advice despite disclaimers, which can have serious consequences.

- **Design Responsibility:** Interfaces should communicate uncertainty and encourage verification rather than presenting outputs as authoritative.

### 5.5 Accessibility and the Equity Gap

The dual-deployment model reveals a structural divide:

- **Resource Inequality:** Users with high-performance hardware can run local models for more privacy and control, while others must rely on cloud solutions that involve costs or data exposure.

- **Technical Barriers:** The system assumes a level of technical skill that can exclude non-expert users or those who rely on assistive technologies.

### 5.6 Open-Source Ecosystem Impact

- **Transparency and Collaboration:** Open-source availability allows auditing, extension, and community-driven improvements.

- **Deployment Risks:** Anyone can deploy the system without safeguards, which may violate ethical or privacy norms and emphasizes the need for responsible defaults and clear guidelines.

## 6. Recommendations & Future Work

### 6.1 Technical Recommendations

- **Hybrid Indexing Pipeline:** The current indexing pipeline is working through embedding chunks into vectors to store in a vector DB. It also processes documents sequentially, but we can consider utilizing parallel tracks to optimize it for structured files such as CSV, JSON, spreadsheets and a separate track for unstructured documents such as PDFs, DOCX, and images to reduce indexing time. This also allows for format-specific preoprocessing steps to run independently without blocking one another. Additionally, introducing page-based indexing for multi-page documents like PDFs would allow chunks to be anchored to specific page numbers, giving more precise citations without the need for a complete vector search, reducing complexity for PDFs.
- **Adopt a scalable Vector Database for larger requirements:** Currently utilizing sqlite-vec would perform flat O(n) kNN scans, which is relatively fast for local file collections but would degrade with large files as the number of chunks would grow. A possible recommendation would be to introduce an optional adapter on top of the approximate nearest neighbour indexed store like QDrant or LanceDB, as it would exist behind the existing retrieval interface but operate for larger document sets.
- **Better reranking:** The current reciprocal rank fusion is strong, but considering something like a cross-encoder reranker as a post-retrieval step would help evaluate and produce more relevant scores for ambiguous or multi-part queries to ensure the response + citations match with the user query (this would be an additional step to the retrieval pipeline).
- **Metadata-aware chunking:** Fixed-size chunking would lose structure for files like spreadsheets and JSON, where the rows and nested fields would carry a relational meaning. This means the

chunking algorithm should respect the format boundaries of the file. An example of this would be when splitting DOCX files at headings and keeping CSV rows atomic. Also, considering metadata of the actual file itself as part of the chunking process itself, such as a metadata chunk, would be a great additional step to add more information on that file when indexing.

- **PDF/table-aware parsing:** Tables inside PDFS are currently treated as flat text after extraction, which causes the column relationships to break across chunking. As mentioned above, this needs to be atomic in the sense that the relations of the table would be preserved. This would require a dedicated table parser to preserve row/column structure before chunking in order to significantly improve retrieval accuracy on document types with tables.
- **Better benchmark automation:** Currently the benchmarking system tests for a variety of datasets for accuracy, query speed and citation accuracy, but adding more datasets and prompts to benchmark over would provide more meaningful data. Improving the automation to scale quickly over the datasets and gather more data insight regarding the pipeline would also be beneficial.

## 6.2 Known Limitations

- **SQLite scaling limits:** The query latency scales linearly with the indexed chunks, where large collections would require a database migration and not just configuration changes.
- **Single-user local-first constraints:** The filewatcher and SQLite database are not designed for concurrent multi-process access, as it would make the system unsuitable for environments beyond a single user space (e.g., a team using a shared filesystem).
- **Citation drift edge cases:** When there are files with similar names in different directories but with variation in context, it may cite the wrong file, as this would require additional ways to differentiate between the files (beyond just metadata and context).
- **Cross-document reasoning limitations:** Answering queries that may require answers beyond a single document, such as comparisons between two documents. This requires a deeper research or graph-level guidance for the LLM to conduct deeper research and answer queries.

## 6.3 Future Work

- **Cloud-sync optional mode:** Allowing users to sync their SQLite index to a personal cloud storage of their choosing to enable cross-device access, while the user is in control of their own storage. This would require updating the UI to allow users to pass in credentials to sync their storage control options seamlessly.
- **Multi-user collaboration:** A shared workspace mode would require a server-hosted database and adding per-user file scope access controls. Additionally, this would involve handling race conditions between multiple file changes that the watcher would need to appropriately handle.
- **advanced citation visualization:** Page and paragraph coordinates are recoverable at ingestion time for PDFS and DOCX files, which we can use to our advantage, as if we store this alongside chunk metadata, we can extract this at the UI level and highlight exact passages a citation maps to rather than just citing source file. Additionally, adding a 2D and or 3D visualization of chunks connected per file if a user wants to see a visualization of chunks brought into context would be an interesting concept to visualize (like a brain of chunks per file).

- **Evaluation dashboards and Model Comparison:** Since the system supports switching between Ollama and cloud APIs, it would be a nice integration to support a comparison between model answers, citation sets, latency, and confidence on the same query to compare model performance on the pipeline. Additionally, integration of the evaluation data gathered into clean UI dashboards would also be super useful for clean visualization of data metrics for evals.
- **Electron Mode:** The Electron mode is an experimental mode where we launch its open application instance instead of it being a webpage on the browser itself. This desktop app would significantly lower the barrier to entry for non-technical users who cannot run a local server from the command line. Additionally, it's viewable as a separate desktop app, meaning it would be easier to find in the open apps rather than searching through multiple tabs in a browser for the application. Additionally, there's also a file search mode in the electron mode where it searches for files similar to that of macOS Spotlight, allowing for quicker file searching without the need for complex queries.

# 7. Conclusion

Document query systems fail to meet user needs when they generate unverifiable responses. Standard LLMs hallucinate without grounded context and native filesystem tools cannot reason semantically. Intelligent Document Reference resolves this by implementing a local-first RAG pipeline that indexes user documents in a lightweight sqlite-vec vector store, retrieves content through hybrid semantic and lexical search, and generates responses that are strictly grounded in source citations.

Benchmark evaluations across 16 dataset suites confirms that the system achieves stable retrieval performance and high semantic response quality, with the divergence found between keyword and judge scores attributed to the paraphrasing behaviour rather than factual error. The citation-grounding mechanism acts as the main trustworthy mechanism where every answer is traceable to a specific chunk and source file which enables users to verify AI reasoning themselves.

The system’s modular architecture with interchangeable embedding models, configurable retrieval parameters, and file parsers make it a practical foundation for future research into local-first, privacy preserving document intelligence. Future extensions around scalable indexing, multi-user access, and upgraded citation visualizations are what make this project a reusable open-source platform that can spark innovation.

## References


Bianchini, F. (2025). Retrieval-Augmented Generation. In: De Luzi, F., Monti, F., Mecella, M. (eds) Engineering Information Systems with Large Language Models. Springer, Cham. https://doi.org/10.1007/978-3-031-92285-5_7

Brown, A., Roman, M., & Devereux, B. 2025. A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges. Big Data Cogn. Comput. 2025, 9(12), 320; https://doi.org/10.3390/bdcc9120320

Bruch, S., Gai, S., & Ingber, A. (2023). An analysis of fusion functions for hybrid retrieval. ACM Transactions on Information Systems, 42(1), 1–35. https://doi.org/10.1145/3596512

OpenAI. (2026, April 5). *ChatGPT (GPT-5) [Large language model]*. https://chatgpt.com/share/69d2ba80-fbd4-8330-a15e-343d52a1f228

OpenAI. (2026, April 5). *ChatGPT (GPT-5) [Large language model]*. https://chatgpt.com/share/69d3f889-b7f0-8333-96cc-e42ab779136b

Weinberg, Abraham. (2026). Hybrid Dense-Sparse Retrieval for High-Recall Information Retrieval. https://doi.org/10.13140/RG.2.2.23909.46562